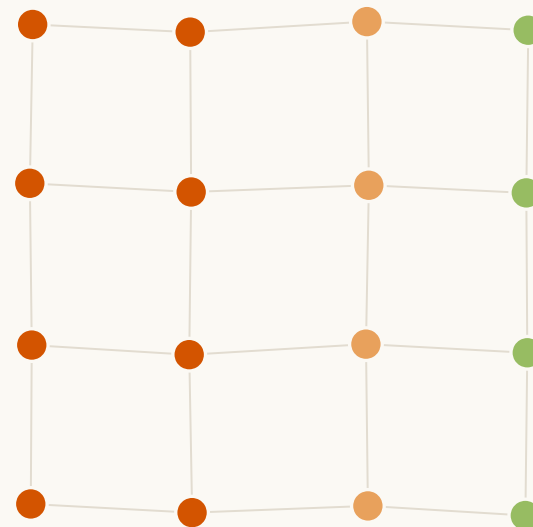


제24회 임베디드SW경진대회 · 자유공모 부문

불사

팀 화중생련 火中生蓮

고장(노드 파괴)을 데이터로 바꾸는 자가치유 산불 감시 메시 네트워크



재난에선 노드 파괴가 '필연'이다

산불·지진·붕괴 현장에서 센서 노드는 **반드시 죽는다**. 기존 시스템은 이 죽음을 단순 '손실'로 처리한다. 우리는 **죽음의 시공간 패턴 자체를 데이터로 승격**해, 사라진 노드들이 오히려 불의 방향·속도를 그리게 한다.

문제 정의

위성·드론은 연기·수목 아래 지표 화선을 못 본다. 진화 지휘·대원 안전이 직결된 실시간 공백이 남는다.

기존의 공백

순정 painlessMesh는 죽은 노드를 우회만 할 뿐, 그 죽음에 담긴 정보를 버린다.

우리 목표

고장 패턴을 데이터로 + 자가치유로, 그 공백을 지상에서 실시간으로 채운다.

→ 응용(산불)이 아니라 아키텍처(고장→데이터 + 자가치유)에 독창성이 있다.



상공이 못 보는 '지표 화선'의 실시간 공백

무엇이 안 보이나

연기와 수목 캐노피 아래에서 실제로 번지는 지표 화선(fire front)의 위치·방향·속도. 위성은 광역 개요만, 드론은 상공·간헐적이라 초 단위 국지 변화를 놓친다.

왜 치명적인가

진화 지휘부의 자원 배치와 최전선 대원의 대피 타이밍이 바로 이 정보에 달려 있다. 공백이 곧 인명·재산 피해로 이어진다.

현장 동기 (2025 의성 산불)

대형 산불 대응 현장에서, 지상의 실시간 화선 정보가 없어 판단이 지연되는 국면을 보며 '지상에서 화선을 실측하는 저가 레이어'의 필요를 절감했다.
(동기·삽화로만 사용 — 반사실 주장 아님)



고장을 '손실'이 아니라 '데이터'로

① 노드 파괴

불에 타 죽는 순간

② 죽은 시각·좌표

= 불이 도착한 시각

③ 방향·속도·ETA

+ 신뢰도 등급

목표 1 — 고장의 데이터화

교차검증된 '죽음'을 확정 이진 신호로 모아, 흩어진 (좌표·죽은시각)에서 화선의 위치·방향·속도를 추정한다.

목표 2 — 자가치유

노드가 죽어 경로가 끊겨도 남은 노드들이 자동으로 우회로를 다시 짜, 관측 자체가 계속되게 한다.

→ 두 목표가 합쳐져: 노드는 죽지만(불에 타도) 시스템의 관측은 '죽지 않는다'. = 불사(不死)의 이중 의미.



하드웨어 · 소프트웨어 구성

하드웨어

- 센서 노드: ESP32 × 16 (WiFi 메시)
- 온도 감지: DS18B20 디지털 온도센서 (9비트 고속 모드)
- 상태 표시: WS2812 LED (생사·경보 시각화)
- 게이트웨이: 라즈베리파이 (메시↔대시보드)
- 전원(데모): USB 5V 급전 — 12분 연속 구동과 회차 간 재현성 확보
- 전원(배치): 배터리 + 승압 모듈(MT3608)

16노드 실물 재료비 ≈ 45만 원 (저가 소모형 설계).

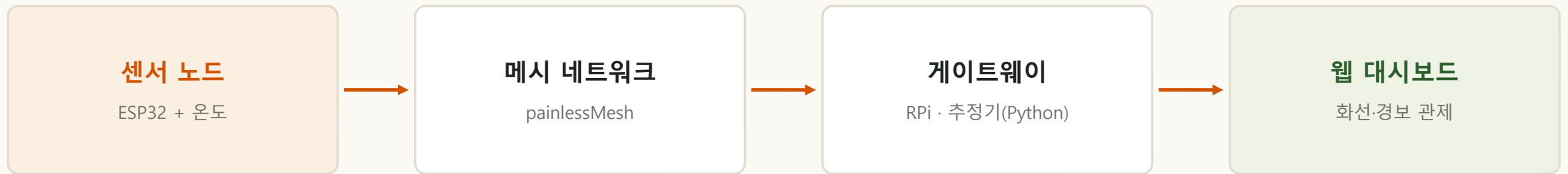
※ 열풍기·변압기 등은 검증용 계측 장비로 노드 원가와 별개.

소프트웨어

- 노드 펌웨어: C/C++ (Arduino-ESP32 + painlessMesh)
— 메시 자동 구성·우회 + 메시 시각 동기 제공
- 게이트웨이: Python — 시물 추정기 코드 그대로 재사용
- 관제: 자체완결 웹 대시보드 (HTML, 서버 불필요)
- 개발·검증: 네트워크 시뮬레이터(Python), Wokwi(컴파일 확인)
- 판정 일원화: 화재/비화재 판별을 단일 함수로 — 시물과 실물이 같은 코드를 호출한다(시물 결과가 실물의 근거가 되도록).



데이터 흐름 아키텍처



노드가 임종신호·하트비트를 메시로 흘리면, 게이트웨이가 죽은 노드의 시공간 패턴에서 화선을 추정하고, 대시보드가 생사·자가치유 경로·참/추정 화선·대피경보를 실시간 재생한다.

핵심 이식성 증명 · 판정 일원화

게이트웨이가 시뮬레이터의 추정기 코드를 그대로 재사용 → 모의 데이터에서 시뮬 직접값과 완전 일치. 나아가 화재/비화재 판별도 단일 함수로 통합해 시뮬·실물이 같은 판정을 쓴다 — 두 경로가 다른 판정을 쓰면 시뮬 결과가 실물의 근거가 되지 못하기 때문.

시스템을 이루는 네 개의 핵심 모듈

①

임종신호 (Last-Gasp)

노드가 임계온도를 넘겨 죽기 직전 쏘는 마지막 패킷 — 죽음의 시각을 확정.

②

자가치유 라우팅

노드가 죽어 길이 끊기면 sink 기준 역방향 BFS로 우회로를 자동 재계산.

③

오탐 방어 (교차검증) ★

통신두절 vs 진짜 파괴를 다중이웃 + 온도로 구분 — 순정
painlessMesh가 못 하는 계층.

④

도착시각장(場) 추정 ★

죽은 (좌표·시각)에서 최소제곱으로 방향·속도·ETA와 신뢰도를
역산.



죽은 노드의 (좌표·시각) → 불의 방향·속도·ETA

각 노드가 타 죽은 시각 = 불이 그 지점에 도착한 시각. 흩어진 (x, y, 죽은시각)에 면을 맞추면 그 기울기가 곧 화선이다. 실제 구현은 **2단 구조**
— 노드마다 국소 적합을 돌리고, 그 결과들을 신뢰도로 가중해 합친다.

1단 · 국소 평면 적합

노드 j마다 독립적으로:

- 무선 이웃 n 창 안의 사망들을 모아
- 최소제곱으로 평면을 맞추고
- 기울기 ∇T_j 를 얻는다

→ 노드 수만큼 N개의 추정치

2단 · 집계

N개를 하나로 합친다:

- 방향 — 신뢰도 가중 벡터합
가중치 $w = |\nabla T|^2$
 - 속도 — 중앙값(강건 통계)
 - 진단 — 유효표본수 n_{eff}
- 두 출력이 다른 집계를 쓴다

출력

- 진행 방향 $\hat{n}$
- 진행 속도 s
- 지점별 도착예상시각 ETA
- 신뢰도 등급

정보가 부족하면 값을 내지 않고
INSUFFICIENT를 반환한다.

$$\nabla T = \hat{n} / s \rightarrow \text{방향 } \hat{n} = \nabla T / |\nabla T|, \text{ 속도 } s = 1 / |\nabla T|$$

가벼운 최소제곱 — 저사양 게이트웨이에서 즉시 계산 (딥러닝 아님)

모든 노드에 똑같이 걸리는 센서 지연은 평면의 절편에만 흡수되어 기울기에서 상쇄된다 — **방향·속도가 센서 지연에 원리적으로 면역인 이유(수식 증명).**



'통신두절'인가, '진짜 파괴'인가

노드가 조용해졌다고 다 죽은 게 아니다. 잠깐 끊긴 것(통신두절)이나 불이 아닌 죽음(배터리·고장)을 화재로 오판하면 가짜 화선이 그려진다. 세 층으로 막는다.

1층 · 다중 이웃 교차검증

한 노드가 침묵해도 즉시 죽음으로 보지 않는다. K개 이웃(K_confirm=3)이 독립적으로 '연결 끊김'을 확인해야 사망 후보로 승격 — 일시적 링크 노이즈를 걸러낸다.

2층 · 온도 교차검증

사망 후보가 직전에 고온을 겪었는가? 화선에 의한 죽음이면 온도 상승이 선행한다. 온도 정황이 뒷받침될 때만 '화선 사망'으로 확정.

3층 · 임종신호 (비대칭)

임종신호는 있으면 화재 증거, 없어도 기각 근거가 아니다 — 전소로 신호조차 못 보낸 노드가 바로 진짜 화재이므로. 이 비대칭성은 테스트 2개로 고정.

설계 원칙 — 판별은 정보가 가장 많은 곳에서

말단 노드의 게이트는 **의도적으로 느슨하게** 두고, 최종 판별은 이웃 정보가 모두 모이는 게이트웨이에서 한다. 말단에서 조이면 게이트웨이가 불 표본 자체가 굵기 때문. **결과: 측정 전 구간(통신두절 0.30까지) 오탐률 0%, 비화재 COOL 누수 86.4% → 0%.**



길이 끊겨도 관측은 죽지 않는다

자가치유 라우팅

- 브리지(id 99) 기준 역방향 BFS로 각 노드의 상행 경로를 구성.
- 노드가 DEAD로 확정되면 영향 구역만 경로를 즉시 재계산.
- 연결성이 유지되는 한, 죽은 노드를 우회해 데이터가 계속 sink에 도달.

· 실물 4노드 메시 실측 (2026-08-30)

중계 노드 이탈 → 데이터 전달 복구 26.0초 / 노드 재합류 3.0초
트리는 원래 형태로 되돌아가지 않고 다른 유효한 형태로 수렴했다.
※ 시물의 100ms는 틱값(성능 아님).

임종 신호 (Last-Gasp)

- 노드가 임계온도를 넘어 죽기 직전, 마지막 패킷을 방출.
- '언제 죽었는가'를 확정 → 도착시각 추정의 입력 시각을 정밀화.
- 하트비트(주기 신호) 누락과 결합해 침묵을 판정.

죽음의 '시각'이 정확할수록 화선 방향·속도 추정이 정밀해진다.

자가치유(관측 지속) + 임종 신호(죽음의 시각 확정)가 만나, '죽는 노드'가 '살아있는 정보'가 된다.



코드 구성과 개발 규율

저장소 구성

- sim/ — 시뮬레이터 · 추정기(estimator) · 집계(aggregate)
- firmware/ — ESP32 노드 펌웨어(C/C++)
- gateway/ — 라즈베리파이 추정기(Python, sim 재사용)
- dashboard/ — 자체완결 웹 관제
- docs/ — 연구노트(PROGRESS · DECISIONS · 알고리즘 명세)

결정 로그(D-001~)를 남겨 '왜 그렇게 정했는가'를 전부 추적 가능하게.

추정기 동결 + 버전 분기

- 핵심 추정기 파일을 동결(frozen) — 모든 개선은 버전 분기로만.
- 그래서 어떤 실험도 과거 결과를 훼손할 수 없다.
회귀가 매번 비트 단위 동일함을 확인(최대차 0.000e+00).
- 자동 테스트 54건 상시 통과.
- 결과가 나빠도 파라미터를 유리하게 고치지 않는다(기록만).
→ 데모에만 맞춘 과적합을 피한다.

→ 값을 정하기 전에 먼저 재고,젠 값에서 파라미터를 유도한다. 완성도는 '보기 좋은 숫자'가 아니라 '재현 가능하고 정직하게 측정된 숫자'에서 나온다.

하드웨어 없이 먼저 검증하고, 정직하게 교정

장애 · 하드웨어 리스크

부품 도착 전 알고리즘을 못 짜면 개발이 지연되고, 실물에서 처음 검증하면 리스크가 크다.

해결 · 시물 선(先)개발

네트워크 시뮬레이터로 4대 기능을 먼저 구현·검증. 정상 조건에서 방향 2.12°·속도 0.10%·전달률 92.3%·오탐 0% 확보.

장애 · 가짜로 좋은 수치

초기엔 바람이 고정 파형이라 시드를 바꿔도 오차가 안 변해 예러바가 가짜로 0에 가까웠다. 좋아 보이는 숫자가 사실은 측정이 아니었다.

해결 · 사전등록 제도

바람을 시드마다 다른 실현으로 바꿔 진짜 통계로. 나아가 물리 예측을 미리 등록하고 맞든 틀리든 전부 기록하는 규칙을 세웠다 — 기각된 예측 5건이 문서에 그대로 남아 있다(단조성·취약성·곡률·채점·ETA 게이트).



방향오차 82° — 진짜 원인을 찾기까지

타원 화선의 측면 조건에서 방향오차가 82~86°. 90°가 무작위 추측의 한계이므로 사실상 아무 정보도 못 주는 상태였다. 세 번의 진단이 전부 추정으로 기각된 뒤에야 원인에 닿았다.

	가설	검정 방법	결과
①	곡률 탓 — 화선이 굽어 평면 근사가 깨짐	창을 넓혀 곡률 영향을 분리	기각 — 창을 넓히자 81.8° → 5.6°
②	채점 방식 탓 — 국소 법선 vs 전역 축 혼동	채점 코드를 직접 확인	기각 — 이미 국소 법선 기준이었다
③	표본 부족 탓 — 창 안에 이웃이 모자람	표본 수·조건수 진단	부분 기각 — 13개인데도 실패
④	집계 가중치가 원인	노드별 개별 오차와 가중치 분포를 분해	★ 확정

+0.986

corr(속도, 방향오차)

속도가 큰 노드가 곧 틀린 노드

86.4%

한 노드의 가중치 비중

그 노드의 개별 오차 85.21°

1.35°

개별 오차의 중앙값

나머지 노드들은 맞히고 있었다

정답은 표본 안에 있었다 — 집계가 그것을 버리고 최악의 노드 하나를 골랐다. 가중치가 $w = 1/|VT|$ (불확실할수록 크게)로 부호가 반대였다. 오차 전파식에서 다시 유도해 $w = |VT|^2$ 로 교정. 82~86°는 시물 타원 화선의 측면 조건에서 창 8초·결함 가중치로 낸 전역 방향 오차다. 같은 추정을 20cm 실물 격자·창 327초에서 국소 법선 기준으로 재면 이웃이 8개인 내부 노드는 0.33~1.17°, 이웃이 3~5개인 가장자리·모서리는 0.96~33.20°다. 남은 오차는 알고리즘이 아니라 이웃 배치에서 온다.

교정 뒤 방향오차 — 직선 화선 0.03°, 타원 정면 0.88°. 82~86°가 나왔던 타원 측면·노드 희소 조건은 교정 뒤에도 값을 내지 않고 자기 차단한다. 그 조건의 방향오차는 미측정이다.



가중치를 고쳐도, 정보가 원래 없는 조건은 남는다

희소한 측면 조건에서는 창 안에 쓸 만한 관측 자체가 없다. 어떤 알고리즘도 못 본다. **그때 시스템은 무엇을 해야 하는가?**

집계 방식	출력	판정
기존(가중 결합)	85.05°	표본 부족을 감지하지 못하고 값을 출력
균일 가중	84.90°	집계 방식을 바꿔도 결과는 같다
중앙값	84.90°	집계 방식을 바꿔도 결과는 같다
역분산 (채택)	값 없음 — INSUFFICIENT	★ 표본 부족을 감지해 판정을 보류

설계 결정 — 폴백을 넣지 않는다

산불 대응에서 '모른다'와 '정반대를 확신한다'의 비용은 대칭이 아니다. 방향을 모르면 판단을 보류하지만, 90° 틀린 방향을 믿으면 대원을 화선 쪽으로 보낸다.

실제 로그에는 "냈다면 틀렸을 것: True"가 함께 남는다.

진단량 · 유효표본수 n_{eff}

표본이 13개여도 하나가 가중치를 독식하면 n_{eff} 는 1.33까지 떨어진다.

· 정상 12.7~13.0 · 가중치 독식 1.33~1.74 · 진짜 표본 굶주림 1.04

원표본 수와 짝지어 표기해야 두 실패가 구분된다.

★ 같은 원칙을 우리 계측 도구가 어겼다 — 센서를 못 찾은 로거가 오류 한 줄 뒤 영구히 침묵해, 배선을 고쳐도 정상과 구별되지 않았다. 침묵 대신 상태를 계속 출력하도록 고쳤다.



시물에선 보이지 않는 실물 전용 위험 셋

① 시계 동기

서로 다른 노드의 사망 시각을 빼서
기울기를 만드는 알고리즘이므로
시간축이 같아야 한다.

초기 펌웨어는 보드 부팅 기준 시계를
썼다 — 보드마다 원점이 달라 뺄셈
자체가 무의미해지는 원리적 파탄.

→ 메시 동기 시각으로 교체. 사망 시각도
'루트의 확정 시각'에서 '당사자 노드가
각인한 시각'으로 (확정 시각엔 감지·투표
지연이 섞여 있었다).

② 센서 열관성 τ

센서는 공기 온도를 즉시 따라가지 못한다.
이 지연이 곧 사망 시각의 오차가 된다.

시물 시험 범위 0 ~ 10초
실측 범위 11 ~ 92초

→ 검증했다고 믿은 범위 밖에 실재가
있었다.

★ τ 는 온도가 아니라 유속의 함수 — 같은
센서·같은 시행 안에서 제트 21.5초 / 실내
92초로 4.1배 차이. 바뀐 것은 공기가
움직이느냐뿐이었다.

★ 노드별 편차 σ_τ 는 '미측정'으로 확정. 한때 5.7%
로 보고했으나 센서 번호가 측정 순서와 교란(corr
-0.903)된 것을 발견하고 우리 손으로 철회했다.

③ 실물 파이프라인 공백

시물에는 화재/비화재 판별이 있는데
실물 경로에는 없었다 — 게이트웨이가
판별기를 아예 호출하지 않고 있었다.

→ 판정 로직을 한 함수로 통합해 시물과
실물이 같은 코드를 호출하도록 교정.

이걸 놓쳤으면 시물에서 검증한 방어가
실물에서 통째로 빠진 채 데모를 했을
것이다.

세 위험 모두 **실물에서 터지기 전에** 발견했다. ①은 코드 검토로, ②는 실측으로, ③은 실물 이식 준비 중에 — 각각 다른 방법이 필요했다는 것이 이 문제들의 성격을 보여준다.



'자기 차단'은 정보 부족이었나, 우리 상수였나

우리는 「정보가 없으면 값을 내지 않는다」를 강점으로 보고했다. 그런데 알고리즘을 20cm 실물 격자로 옮겨 다시 재보니, 16노드 전부가 값을 내지 못했다. 자랑하던 기능이 전면 작동한 것인지, 아니면 무언가 잘못된 것인지 구분해야 했다.

창 크기 dt_window (초)	산출 성공	전역 방향	참값 55.67° 대비	실패 원인
60	2 / 16	31.64°	-24.03°	전부 표본 부족
120	12 / 16	51.12°	-4.55°	전부 표본 부족
173	15 / 16	55.76°	+0.09°	표본 부족 1건
240	16 / 16	55.89°	+0.22°	없음
327	16 / 16	55.92°	+0.25°	없음 ← 유도값

8 s

설정된 창

이웃 통과 182 s · 반경 327 s

0 건

기하(공선) 실패

실패는 전부 '표본 부족'이었다

4 건

규모 의존 상수

dt_window · residual · alert · speed

dt_window = 8.0 s 는 시물 화선 1.5 m/s 기준값이었다. 20cm 벤치의 화선은 1.1 mm/s — 이웃이 한 명도 창에 못 들어와 모든 국소 집합이 자기 자신 하나가 됐다. 값을 새로 고르는 대신 상수를 물리에서 유도하도록 구조를 바꿨다 — $dt_window = radio_range / v_front_expected$. 시작 로그에 유도식과 계산값을 함께 찍는다.



고장을 '손실'이 아니라 '데이터'로

비교 대상	그들의 방식	한계	우리 (불사)
온도 스트리밍	살아있는 노드가 온도 상시 전송	초저가 대량 노드엔 상시 전송 부담(전력·대역폭)	교차검증된 '죽음'을 확정 이진 신호로 → 화선 위치·방향·속도
Dryad Silvanet	불 오기 전 조기 감지(내열, +85℃ 한계)	불 속에선 못 버팀 — 감지 특화, 화선 추적 아님	타 죽으며 화선을 그림 — 감지(그들)와 추적(우리)은 상보
순정 ESP-MESH	노드 죽으면 경로 우회(연결성만)	죽음을 손실로만 처리, 정보로 안 씀	죽음을 1급 이벤트로 승격 + 도착시각 추정 + 오탐 방어
드론·위성	상공·우주 광역 개요	연기·수목 아래 지표 화선·초 단위 변화 못 봄	지표 실측 레이어 + 지상 통신 백본 — 대체 아니라 상보

★ 헤드라인 차별점 오탐 방어(통신두절 vs 진짜 파괴 구분) = 순정 painlessMesh가 못 하는 우리만의 계층.



정상 성능과 '작동 한계선'을 함께 밝힌다

< 1°

방향오차

직선·정면 조건 (0.03~0.88°)

92.3%

전달률

자가치유 포함

0%

오탐률

통신두절 오판

54

자동 테스트

전건 통과 · 회귀 최대차 0

작동 한계선 (operating envelope) — 어디까지 되고, 어디부터 안 되는가

조건	결과	의미
직선 화선	방향오차 0.03°	정상 운용 영역
타원 화선 · 정면(head)	방향오차 0.88°	데모·실전의 주 조건
강곡률 곡선 화선	방향오차 7.6~13.4°	국소 평면 근사의 한계 — 센서 지연과 무관한 별개 한계
타원 측면 · 노드 회소	자기 차단(값 미출력)	정보 부족과 표본 굽주림을 분리해야 성립 — §4-E
ETA	앵커거리와 상관 0.99~1.00	배치 밀도가 지배 — 알고리즘이 아니라 간격
데모 열원 조건	190.5 °C (실측)	예측 190.0 °C 적중(0.3%). 설정 400·풍량 10·4 cm·체류 8초

모든 노드에 똑같이 걸리는 센서 지연에는 원리적으로 면역 (기울기에서 상쇄 — 수식 증명). 실측에서도 같은 성질이 재현됐다 — 센서 위치가 흔들려 도달 온도가 16.5°C 변하는 동안 τ 는 $-0.061\text{ s/}^{\circ}\text{C}$ 로만 반응했다.

※ 자기 차단은 유효한 기능이지만, 초기 보고에서 '정보 부족'으로 해석한 것 중 일부는 우리가 설정한 창 크기가 만든 표본 굽주림이었다. 원인 분해와 재발 방지는 §4-E.



누가 쓰고, 어떻게 배포하나

수요자 1 · 진화 지휘부

지표 화선의 실시간 위치·방향·속도로 '어디에 자원을 먼저 투입할지' 판단. 자원 배치 정확도 ↑.

수요자 2 · 최전선 대원

'이 구역에 화선 N분 내 도달' 대피 타이밍 제공. 대원 안전과 직결 (속도는 안전마진으로 보수적 경보).

배포 · 고위험 구역 사전 배치

상습 발화지, 도시·마을 인접 산림(WUI), 주요 시설·문화재·등산로에 핀포인트. 초저가라 촘촘히 깔아도 부담 적음(사전 배치는 상용 선례로 검증).

기대효과

상공 관측이 놓치는 지표 화선을 지상에서 실시간화 → 자원 배치 정확도와 대원 안전을 동시에 끌어올린다.

저가 소모형의 경제 논리와 확장성

비용 논리

- 노드 개당 수천 원 → 불 속에 들어가는 역할엔 내구성을 '살' 수 없으니 저가 소모형이 유일하게 합리적.
- 16노드 실물 ≈ 45만 원.
- 예방 투자 vs 산불 1회 피해(인명·산림·재산) = 후자가 압도적.
- 불날 때마다 갈아끼우는 모델이 오히려 경제적.

발전 가능성

- 같은 '고장-패턴-데이터화 + 자가치유' 구조를 실내 화재·가스 누출·구조물 붕괴·홍수로 확장 가능.
- 온도 상승 궤적·곡선-허용 시공간 정합·다중소스 추정(#2e)이 다음 연구 축.
- 드론·위성과 대체가 아니라 상보 레이어로 통합.
- 전력: 상시 메시 구조상 노드당 90~130 mA [추정 — 데이터시트 기준]. 저전력 모드(평시 deep sleep · 화재 시 메시 기동)는 향후 과제.

→ 독창성은 특정 응용이 아니라 '고장을 데이터로 승격하는 아키텍처'에 있어, 재난 전반으로 이식된다.



1인 개발자로 전 영역을 단계적으로 수행

단계	기간	내용	상태
① 시뮬·알고리즘	7월	시뮬레이터 · 도착시각 추정 · 스트레스 시험	● 완료
② 알고리즘 정밀화	8월 상순	집계 결함 교정 · 자기 차단 도입 · 명세 대조	● 완료
③ 실측·실물 포팅	8월 중순	센서 열관성 실측 · 펌웨어 포팅 · 메시 실증	● 완료
④ 자가치유 실증	8/29~8/30	4노드 메시 · 중계 이탈/복귀 실측	● 완료
⑤ 조립·교정	8/30	16노드 조립 · 감지부 좌표 · 열원 교정	● 진행 중
⑥ 리허설·촬영	8/31~9/1	전 구간 리허설 · 시연 촬영	● 예정
⑦ 문서·제출	9/2~9/3	영상 편집 · 보고서 완성 · GitHub 공개	● 예정

1인 업무 분장 (영역별)

펌웨어(ESP32·painlessMesh) · 알고리즘/게이트웨이(Python 추정기) · 하드웨어(16노드 조립·데모·계측) · 문서/발표(보고서·영상)를 단독으로 단계적 수행. 시뮬 선개발로 리스크를 앞당겨 해소하고, 추정기 동결·사전등록 규율로 범위를 통제 — 1인 개발의 범위 관리·완결 역량을 보인다.

